

Informe final — Agente RAG DNI

Asistente Inteligente de Conocimiento sobre Damos Nuestra Ilusión

Asignatura: Inteligencia Artificial **Grado:** Tecnologías Interactivas — Universitat Politècnica de València

Integrantes: Javier Serrano Marco, Javier Camarena Cuartero y Jaime Ferrer Prats **Curso académico:** 2025-2026

1. Introducción

El objetivo de esta práctica ha sido desarrollar un agente inteligente capaz de responder preguntas sobre la asociación **DNI (Damos Nuestra Ilusión)** a partir del corpus oficial proporcionado. El sistema no pretende responder usando conocimiento general del modelo de lenguaje, sino recuperar información relevante del corpus, generar una respuesta apoyada en dichas fuentes y rechazar aquellas preguntas cuya información no se encuentra disponible.

La solución desarrollada implementa un sistema **RAG (Retrieval-Augmented Generation)**. Este enfoque combina un mecanismo de recuperación documental con un modelo generativo: primero se buscan los fragmentos más relacionados con la consulta y, posteriormente, el modelo redacta una respuesta limitada al contexto recuperado.

El proyecto ha evolucionado desde la base inicial hasta una solución con arquitectura hexagonal, retrieval híbrido, benchmark comparativo de cuatro modelos y evaluación avanzada mediante RAGAs y métricas propias.

2. Objetivos de la solución

Los objetivos principales del sistema son los siguientes:

- Responder preguntas sobre DNI utilizando únicamente información presente en el corpus.
- Citar los archivos fuente utilizados para fundamentar cada respuesta.
- Evitar alucinaciones mediante un rechazo explícito de preguntas fuera de ámbito.
- Gestionar correctamente contradicciones reales existentes entre documentos.
- Permitir sustituir modelos, embeddings o vector stores sin modificar la lógica central.
- Comparar distintos modelos generativos mediante un benchmark reproducible.
- Evaluar la calidad del sistema con métricas automáticas y revisión manual.
- Incorporar un frontend Streamlit funcional para facilitar la demostración oral del agente.

La entrega declara las bandas 5, 6, 7, 8 y 10, correspondientes al pipeline funcional, cita de fuentes, benchmark con cuatro modelos, evaluación RAGAs con métricas propias y arquitectura hexagonal. Además, declara el extra de frontend Streamlit funcional.

3. Corpus y problemática del dominio

El agente trabaja sobre los **16 documentos oficiales de DNI** incluidos en la práctica. El corpus reúne información sobre la asociación, su filosofía y sus principales proyectos sociales, como desayunos solidarios, actividades con personas mayores y refuerzo escolar.

Una dificultad importante es que los documentos no presentan una estructura uniforme. Algunos contienen texto narrativo, otros recogen preguntas y respuestas con formato **Q:/A:**, y otros contienen horarios o ubicaciones. Por ello, un chunking completamente genérico podía separar preguntas de sus respuestas y perjudicar la recuperación.

Además, el corpus incluye alguna contradicción real. El caso más representativo aparece en el horario de los desayunos solidarios:

- `01_faq_dni.txt` indica que los desayunos son a las 8:00.
- `11_horarios_ubicaciones.txt` indica que suelen realizarse entre las 9:00 y las 12:00.

El sistema no debe inventar cuál de las dos versiones es correcta, sino mostrar ambas e indicar sus fuentes.

4. Arquitectura de la solución

4.1. Decisión arquitectónica

La solución final utiliza **arquitectura hexagonal o ports & adapters**. El objetivo es que la lógica principal del agente no dependa directamente de detalles externos como Ollama, PoliGPT, FAISS o ChromaDB.

El flujo real de ejecución es:

```
consultar.py
  ↓
composition.py
  ↓
ChatbotService
  ↓
Ports del dominio
  ↓
Adapters configurados
```

`consultar.py` actúa como punto de entrada exigido por el contrato. El fichero `composition.py` construye el sistema seleccionando los adapters definidos mediante variables de entorno. El servicio `ChatbotService` contiene la lógica principal del agente y depende de abstracciones, no de proveedores concretos.

4.2. Capas implementadas

La organización principal del código es la siguiente:

```
src/agente_rag/
├── domain/
│   ├── entities.py
│   └── ports.py
```

```
├── chatbot_service.py
├── adapters/
│   ├── llm/
│   │   ├── ollama_llm.py
│   │   └── poligpt_llm.py
│   ├── embeddings/
│   │   ├── ollama_embeddings.py
│   │   └── sentence_transformers_embeddings.py
│   └── retriever/
│       ├── semantic_retriever.py
│       ├── bm25_retriever.py
│       ├── hybrid_retriever.py
│       ├── chroma_vector_store.py
│       └── faiss_vector_store.py
├── composition.py
└── config.py
```

4.3. Ports y adapters

Los ports definen los contratos que necesita el dominio:

- Generación de respuestas mediante un LLM.
- Generación de embeddings.
- Recuperación de fragmentos relevantes.
- Acceso a un vector store.

Los adapters implementados permiten intercambiar tecnologías:

Componente	Adapters implementados
LLM	Ollama, PoliGPT
Embeddings	Ollama, Sentence Transformers
Vector store	FAISS, ChromaDB
Recuperación	Semántica, BM25 e híbrida

Esta decisión permite ejecutar el agente localmente mediante Ollama o probar modelos remotos de PoliGPT sin cambiar el núcleo de la aplicación.

5. Pipeline RAG desarrollado

5.1. Indexación y chunking

El corpus se procesa para construir fragmentos recuperables. Debido a la presencia de documentos con preguntas frecuentes, se incorporó un tratamiento especial para conservar unidos los pares Q:/A:. Así se evita recuperar una pregunta sin su respuesta asociada.

La indexación final utiliza embeddings generados mediante:

```
nommic-embed-text
```

y almacena los vectores en:

```
FAISS
```

El índice real del corpus DNI se construyó mediante:

```
python scripts\build_hexagonal_index.py
```

La indexación generó **281 chunks** del corpus.

5.2. Retrieval híbrido

La recuperación combina dos métodos:

- **Búsqueda semántica**, que identifica fragmentos relacionados por significado.
- **BM25**, que favorece coincidencias léxicas exactas.

Esta combinación resulta adecuada para el corpus DNI, ya que algunas preguntas requieren entender formulaciones similares y otras contienen nombres, ubicaciones o términos exactos que la búsqueda léxica identifica bien.

5.3. Generación de respuestas

Una vez recuperados los chunks relevantes, el agente construye un prompt que obliga al modelo a responder utilizando únicamente el contexto proporcionado.

La salida incluye:

```
{
  "respuesta": "Texto generado por el agente",
  "fuentes": ["archivo_fuente.txt"],
  "chunks": [],
  "metricas": {},
  "trazas": null
}
```

Las fuentes permiten comprobar de qué documentos procede la información utilizada.

5.4. Control de alucinaciones

Cuando la pregunta no puede responderse desde el corpus, el agente devuelve literalmente:

No tengo esa información en mis fuentes.

Este comportamiento se ha comprobado con preguntas fuera del dominio, como:

¿Cuánto cuesta alquilar un piso en Valencia?
¿Qué becas universitarias puedo solicitar este curso?

De esta manera, el agente evita ofrecer respuestas generales o inventadas sobre temas ajenos a DNI.

5.5. Gestión de contradicciones

Para preguntas con información contradictoria en las fuentes, el agente muestra las distintas versiones encontradas y cita los archivos correspondientes.

Por ejemplo, ante:

¿A qué hora son los desayunos solidarios?

la respuesta presenta tanto la versión de las 8:00 como la franja entre 9:00 y 12:00, identificando los documentos que sostienen cada una.

6. Configuración final recomendada

Tras la evaluación realizada, la configuración final seleccionada para el agente es local:

```
LLM_PROVIDER=ollama
LLM_MODEL=qwen2.5:3b
OLLAMA_URL=http://localhost:11434/api

EMBEDDER_PROVIDER=ollama
EMBED_MODEL=nomic-embed-text

VECTOR_STORE_PROVIDER=faiss
FAISS_PATH=./data/dni.index

CORPUS_DIR=./base_conocimiento
VERIFY_SSL=true
```

Esta configuración ofrece varias ventajas:

- No depende de una conexión externa.
- No requiere VPN para utilizar el agente final.
- Evita utilizar claves privadas durante la ejecución normal.

- Mantiene resultados de calidad superiores en la evaluación realizada.

PoliGPT se ha utilizado como alternativa para el benchmark y como juez durante la evaluación RAGAs, siempre mediante clave privada almacenada en `.env` y conexión VPN UPV cuando ha sido necesaria.

7. Benchmark de cuatro modelos

7.1. Diseño experimental

Para comparar modelos generativos se definió un conjunto de **12 preguntas** sobre el corpus DNI. El benchmark incluye:

- preguntas factuales directas;
- preguntas sobre ubicaciones y horarios;
- preguntas sobre documentación necesaria;
- una pregunta con contradicción real entre fuentes;
- dos preguntas fuera de ámbito.

Durante las cuatro ejecuciones se mantuvieron constantes:

- el corpus;
- el chunking;
- el retrieval híbrido;
- los embeddings;
- el vector store FAISS.

La única variable modificada fue el modelo generativo.

7.2. Modelos comparados

Proveedor	Modelo
Ollama local	qwen2.5:3b
Ollama local	llama3.2:3b
PoliGPT	gemma3:27b
PoliGPT	llama3.3:70b

7.3. Resultados del benchmark base

Modelo	Ejecución	Calidad manual	Latencia LLM media (s)	Tiempo end-to-end medio (s)	Tokens/s	Source recall	Fuera de ámbito
qwen2.5:3b	12/12	12/12	5.1858	7.8483	60.4633	0.95	1.00
llama3.2:3b	12/12	10/12	7.9933	10.3742	58.6225	0.95	1.00
gemma3:27b	12/12	12/12	2.2067	4.7592	282.5058	0.95	1.00

Modelo	Ejecución	Calidad manual	Latencia LLM media (s)	Tiempo end-to-end medio (s)	Tokens/s	Source recall	Fuera de ámbito
llama3.3:70b	12/12	12/12	6.0333	8.5075	130.4167	0.95	1.00

7.4. Revisión manual

La revisión manual mostró que tres modelos respondían correctamente a las 12 preguntas:

- qwen2.5:3b
- gemma3:27b
- llama3.3:70b

En cambio, llama3.2:3b falló en dos casos:

Pregunta	Incidencia detectada
q07	Interpretó como contradicción una ubicación para la que sí existía una respuesta principal válida y terminó rechazando la consulta.
q08	Presentó como contradictoria la documentación necesaria para participar y no ofreció la respuesta definitiva esperada.

Estos fallos muestran la importancia de combinar métricas automáticas con revisión cualitativa, ya que el retrieval podía recuperar fuentes adecuadas y, aun así, el modelo generar una respuesta deficiente.

8. Evaluación RAGAs y métricas propias

8.1. Configuración de la evaluación

La evaluación avanzada se realizó sobre los cuatro modelos y las 12 preguntas del benchmark, dando lugar a 48 respuestas evaluadas.

Para calcular RAGAs se utilizó PoliGPT exclusivamente como evaluador:

- **LLM juez:** gemma3:27b.
- **Embeddings de evaluación:** poligpt-embed-bge-m3.

Estos modelos no forman parte del pipeline final elegido, sino que se utilizan únicamente para puntuar las respuestas generadas.

8.2. Métricas RAGAs utilizadas

Se calcularon las cuatro métricas solicitadas:

- **Faithfulness:** mide si la respuesta está respaldada por los contextos recuperados.
- **Answer relevancy:** mide si la respuesta es pertinente respecto a la pregunta.
- **Context precision:** mide la utilidad de los chunks recuperados.
- **Context recall:** mide si el contexto contiene la información necesaria de la referencia.

8.3. Métricas propias

Además de RAGAs, se definieron dos métricas propias:

Expected Source Coverage

Mide la proporción de fuentes esperadas que aparecen entre las fuentes recuperadas por el agente.

Esta métrica es importante porque una respuesta puede parecer correcta aunque se apoye en documentos irrelevantes. En un sistema que debe citar fuentes, recuperar los archivos adecuados es parte esencial de la calidad.

Out-of-Scope Rejection Accuracy

Mide la proporción de preguntas fuera del ámbito DNI que el sistema rechaza correctamente.

Esta métrica evalúa directamente la capacidad anti-alucinación del agente y garantiza que no responda sobre temas no cubiertos por el corpus.

8.4. Resultados RAGAs y métricas propias

Modelo	Faithfulness	Answer relevancy	Context precision	Context recall	Expected Source Coverage	Out-of-Scope Rejection Accuracy
qwen2.5:3b	0.788889	0.638091	0.669444	0.833333	0.95	1.00
llama3.2:3b	0.716667	0.471928	0.669444	0.833333	0.95	1.00
gemma3:27b	0.716667	0.612921	0.669444	0.833333	0.95	1.00
llama3.3:70b	0.775463	0.582995	0.669444	0.833333	0.95	1.00

8.5. Interpretación

Las métricas relacionadas con la recuperación son idénticas en los cuatro modelos:

Context precision: 0.669444

Context recall: 0.833333

Expected Source Coverage: 0.95

Este resultado es coherente con el diseño experimental, ya que el retrieval, los embeddings y el vector store permanecieron constantes en todas las ejecuciones.

También se obtuvo:

Out-of-Scope Rejection Accuracy: 1.00

para todos los modelos. Por tanto, el mecanismo de rechazo funcionó correctamente en las preguntas fuera de ámbito incluidas en el benchmark.

Las diferencias aparecen en la generación de respuestas. `qwen2.5:3b` obtuvo el mejor valor tanto de `faithfulness` como de `answer_relevancy`, además de lograr 12/12 aciertos en la revisión manual. `gemma3:27b` fue claramente el modelo más rápido, pero no alcanzó el mismo nivel de calidad según RAGAs.

9. Modelo seleccionado

El modelo final recomendado para el agente es:

```
qwen2.5:3b mediante Ollama local
```

La decisión se fundamenta en los siguientes resultados:

- Obtuvo 12/12 aciertos en la revisión manual.
- Alcanzó el mejor valor de `faithfulness`: `0.788889`.
- Alcanzó el mejor valor de `answer_relevancy`: `0.638091`.
- Funciona localmente, sin depender de VPN ni de disponibilidad de PoliGPT.
- Mantiene la protección anti-alucinación con un valor de `1.00` en rechazo fuera de ámbito.

Aunque `gemma3:27b` fue más rápido, se considera una alternativa remota orientada al rendimiento, no la mejor opción global para la configuración final.

10. Validación y tests

La solución dispone de tests automatizados sobre el dominio, adapters, configuración e interfaz declarada.

La batería completa se ejecuta mediante:

```
python -m pytest -q
```

El resultado final verificado durante el desarrollo es:

```
54 tests correctos
```

El uso de arquitectura hexagonal permite que muchos tests se ejecuten mediante adapters simulados, sin depender de modelos reales, red externa ni VPN.

Además de los tests automáticos, se realizaron comprobaciones funcionales reales:

Consulta	Comportamiento comprobado
¿Qué es DNI?	Respuesta correcta apoyada en <code>08_preguntas_basicas.txt</code> .
¿A qué hora son los desayunos solidarios?	Presentación de ambas versiones contradictorias y sus fuentes.
¿Cuánto cuesta alquilar un piso en Valencia?	Rechazo correcto por estar fuera del corpus.

11. Frontend Streamlit para demostración

Como extra funcional, se ha incorporado una interfaz web mínima desarrollada con Streamlit. Esta interfaz no sustituye al contrato oficial de corrección, que sigue siendo la función `consultar` expuesta en `consultar.py`, sino que actúa como adaptador de entrada adicional para facilitar la presentación oral del agente.

El frontend permite escribir una pregunta sobre DNI y visualizar la respuesta generada por el agente, las fuentes documentales utilizadas, las métricas de ejecución y las evidencias recuperadas. También permite demostrar de forma clara los dos comportamientos críticos del sistema: el rechazo de preguntas fuera de ámbito y la presentación transparente de contradicciones reales del corpus.

La interfaz se ejecuta mediante:

```
python -m streamlit run streamlit_app.py
```

Durante la validación se comprobó con tres consultas representativas:

Consulta	Comportamiento mostrado en la interfaz
¿Qué es DNI?	Respuesta correcta, fuente <code>08_preguntas_basicas.txt</code> , métricas y evidencias recuperadas.
¿A qué hora son los desayunos solidarios?	Presentación de las dos versiones contradictorias y sus fuentes.
¿Cuánto cuesta alquilar un piso en Valencia?	Rechazo explícito mediante "No tengo esa información en mis fuentes."

12. Dificultades encontradas

Durante el desarrollo se encontraron varias dificultades relevantes.

12.1. Adaptación del repositorio inicial

El repositorio inicial estaba orientado a un caso de ejemplo distinto. Fue necesario sustituir referencias antiguas, adaptar el corpus, revisar la documentación y asegurar que la entrega final describe exclusivamente el caso DNI.

12.2. Contradicciones en las fuentes

La presencia de información diferente sobre los horarios requería evitar una respuesta artificialmente única. Se optó por una solución transparente: mostrar las versiones recuperadas y citar sus documentos.

12.3. Comparación con PoliGPT

Para ejecutar modelos PoliGPT desde fuera del campus fue necesario configurar la VPN UPV y mantener las claves fuera del repositorio. Esto introdujo una dependencia externa únicamente durante el benchmark y la evaluación RAGAs.

12.4. Compatibilidad de RAGAs

La instalación de RAGAs requirió ajustar versiones compatibles de LangChain. Una vez resuelta la compatibilidad, se verificaron individualmente las cuatro métricas antes de ejecutar la evaluación completa.

12.5. Coste temporal de la evaluación

La evaluación RAGAs completa sobre 48 respuestas requirió numerosas llamadas al modelo juez remoto y tuvo una duración elevada. Por ello, se realizó primero una prueba mínima sobre una sola pregunta antes de lanzar el proceso completo.

13. Limitaciones y mejoras futuras

La solución cumple los objetivos planteados, aunque mantiene algunas limitaciones:

- La ejecución de RAGAs depende de PoliGPT y de la VPN UPV fuera del campus.
- La detección de contradicciones puede seguir refinándose para distinguir mejor información realmente incompatible de información complementaria.
- El agente no implementa memoria conversacional persistente.
- El benchmark podría ampliarse con preguntas más ambiguas o que requieran combinar varios documentos.
- El frontend Streamlit está orientado a la demostración oral y no incorpora autenticación, persistencia de sesiones ni despliegue en servidor externo.

Como posibles mejoras futuras se plantean:

- añadir memoria conversacional controlada;
- incorporar reranking de chunks;
- ampliar el conjunto de evaluación;
- evolucionar el frontend Streamlit hacia una interfaz desplegable con historial de consultas;
- estudiar modelos locales adicionales que mantengan calidad con menor latencia.

14. Entregables generados

Los principales ficheros de la entrega son:

Fichero	Contenido
consultar.py	Punto de entrada del agente.
features.json	Declaración de funcionalidades implementadas.
README.md	Guía general de instalación, arquitectura y ejecución.
streamlit_app.py	Frontend Streamlit funcional para la demostración oral.
GRUPO.md	Integrantes y reparto de trabajo.
AI_USAGE.md	Declaración honesta de uso de asistentes de IA.
docs/ARCHITECTURE.md	Explicación técnica de la arquitectura hexagonal.
docs/CONTRACT.md	Contrato de interfaz del agente.
benchmark/benchmark.json	Resultados estructurados del benchmark.
benchmark/benchmark.md	Tabla e interpretación del benchmark.
evaluacion/ragas_results.json	Resultados detallados de RAGAs.
evaluacion/metricas_propias.md	Definición e interpretación de métricas propias.
informe.pdf	Informe final de la práctica.

15. Conclusiones

Se ha desarrollado un agente RAG funcional sobre el corpus oficial de DNI, capaz de recuperar información relevante, citar fuentes, rechazar preguntas fuera de ámbito y manejar contradicciones presentes en los documentos.

La evolución hacia una arquitectura hexagonal mejora la mantenibilidad y la verificabilidad del sistema, ya que permite sustituir componentes tecnológicos sin alterar la lógica central. Esta separación ha facilitado probar adapters distintos, ejecutar el agente con modelos locales y remotos y mantener tests independientes de la infraestructura externa.

La evaluación experimental ha permitido seleccionar de forma justificada `qwen2.5:3b` como modelo final. Aunque PoliGPT ofreció una alternativa más rápida mediante `gemma3:27b`, el modelo local obtuvo mejores resultados de fidelidad y relevancia, además de evitar dependencias externas en la ejecución normal del agente.

En conjunto, la solución desarrollada proporciona un agente documentado, evaluado y defendible, con resultados reproducibles y una configuración final coherente con los objetivos de la práctica.